

```

import crypto from 'node:crypto';
import { q, one, j } from '../db.js';
import { defineCapability } from './policy.js';
import { StepError } from '../drivers/android.js';
import { resolve } from './providers.js';
import { androidWorkspace } from './workspace.js';
import { recordObservation, AUTHORITY } from './canonical.js';

const hash = (v) => crypto.createHash('sha256').update(JSON.stringify(v)).digest(

// Fill {{value}} and {{value.field}} placeholders in a step.
function bind(step, value) {
  const out = {};
  for (const [k, v] of Object.entries(step)) {
    out[k] = typeof v === 'string'
      ? v.replace(/\{\{value(?:\.([a-z0-9_]+))?\}\}\}/gi,
        (_, f) => String(f ? value?.[f] ?? '' : value))
      : v;
  }
  return out;
}

async function mapFor(packageKey) {
  return one(
    `select * from appmap where package_key = $1 and status = 'active'
    order by created_at desc limit 1`, [packageKey]
  );
}

// Drive one app on the business's own device to carry a canonical value into it.
// The write is not believed until the same map reads it back off the screen.
defineCapability({
  key: 'channel.push',
  packageKey: 'kernel',
  summary: 'Carry a canonical value into one installed app on the device.',
  route: 'android',
  sensitivity: 'normal',
  // A deterministic script drives this. No model chooses the steps, and no
  // harness is offered it. The appmap is the whole decision.
  agentSafe: false,
  deterministic: true,
  handler: async ({ ctx, input, executionId }) => {
    const { packageKey, key } = input;
    const ws = await androidWorkspace(ctx.businessId);

```

```

if (!ws) throw new Error('no android workspace');

const app = await one(
  `select * from device_app where workspace_id = $1 and package_key = $2`, [w
]);
if (!app) throw new Error(`${packageKey} is not installed on this device`);
if (!app.logged_in) throw new Error(`${packageKey} is not logged in`);

const map = await mapFor(packageKey);
const routes = j(map?.routes) ?? {};
const route = routes[key];
if (!route?.write) throw new Error(`${packageKey} has no write route for "${k

const value = await canonicalValue(ctx.businessId, key);
if (value === null || value === undefined) throw new Error(`no canonical valu

const executor = await resolve({ slot: 'workspace.executor', businessId: ctx.
if (!executor) throw new Error('no workspace executor bound');
const steps = route.write.map(s => bind(s, value));

try {
  await executor.module.run({ workspace: ws, steps });
} catch (e) {
  if (e instanceof StepError) {
    // The map no longer matches the app. Stop; do not guess.
    await one(
      `insert into repair_item (business_id, package_key, execution_id, step,
        values ($1,$2,$3,$4::jsonb,$5,$6::jsonb) returning *`,
      [ctx.businessId, packageKey, executionId, JSON.stringify(e.step), e.rea
        JSON.stringify(await executor.module.screenshot({ workspace: ws }))]
    );
    throw new Error(`appmap mismatch on ${packageKey}: ${e.reason} (sent to r
  }
  throw e;
}

await q(`update device_app set last_seen = now() where id = $1`, [app.id]);
return { packageKey, key, wrote: value };
},

// Read the field back off the screen, record it as that channel's own
// observation, and let the sync state fall out of the comparison.
verify: async ({ ctx, input }) => {
  const { packageKey, key } = input;
  const ws = await androidWorkspace(ctx.businessId);
  const map = await mapFor(packageKey);
  const route = (j(map?.routes) ?? {})[key];

```

```

if (!route?.read) {
  return { state: 'unknown', evidence: [{ kind: 'note', text: 'no read route'
}

const executor = await resolve({ slot: 'workspace.executor', businessId: ctx.
const { readings, screen } = await executor.module.run({ workspace: ws, steps
const observed = route.assemble ? assemble(route.assemble, readings) : Object

const conn = await one(
  `select * from connection where business_id = $1 and provider_key = $2`, [c
);
await recordObservation({
  businessId: ctx.businessId, connectionId: conn?.id ?? null, source: package
  authorityRank: AUTHORITY.connector, subject: 'business', key, value: observ
});

const canonical = await canonicalValue(ctx.businessId, key);
const inSync = hash(observed) === hash(canonical);

if (conn) {
  await q(
    `update channel_sync_state set channel_hash = $1, canonical_hash = $2,
      in_sync = $3, last_checked = now(), last_pushed = now()
    where connection_id = $4 and key = $5`,
    [hash(observed), hash(canonical), inSync, conn.id, key]
  );
}

return {
  state: inSync ? 'verified' : 'partial',
  evidence: [
    { kind: 'screen_read', key, observed },
    { kind: 'canonical', value: canonical },
    { kind: 'screenshot', screen },
  ],
};
},
});

// Read what an app currently shows, without changing anything.
defineCapability({
  key: 'channel.read',
  packageKey: 'kernel',
  summary: 'Read what one installed app currently shows for a canonical key.',
  route: 'android',
  // Read-only. Safe to hand to a model, because it cannot change anything.
  agentSafe: true,

```

```

handler: async ({ ctx, input }) => {
  const { packageKey, key } = input;
  const ws = await androidWorkspace(ctx.businessId);
  const map = await mapFor(packageKey);
  const route = (j(map?.routes) ?? {})[key];
  if (!route?.read) throw new Error(`${packageKey} has no read route for "${key}"`);

  const executor = await resolve({ slot: 'workspace.executor', businessId: ctx.businessId });
  const { readings } = await executor.module.run({ workspace: ws, steps: route.steps });
  const observed = route.assemble ? assemble(route.assemble, readings) : Object.fromEntries(readings);

  const conn = await one(
    `select * from connection where business_id = $1 and provider_key = $2`, [ctx.businessId, packageKey]
  );
  await recordObservation({
    businessId: ctx.businessId, connectionId: conn?.id ?? null, source: packageKey,
    authorityRank: AUTHORITY.connector, subject: 'business', key, value: observed
  });
  return { packageKey, key, observed };
},
verify: async ({ result }) => ({
  state: 'verified',
  evidence: [{ kind: 'screen_read', observed: result.observed }],
}),
});

function assemble(shape, readings) {
  const out = {};
  for (const [field, source] of Object.entries(shape)) out[field] = readings[source];
  return out;
}

export async function canonicalValue(businessId, key) {
  const row = await one(
    `select value from business_fact where business_id = $1 and key = $2 and effective_date = $3`
  );
  return j(row?.value) ?? null;
}

```